

Comprehensive Project Guide: Student Mental Health Detection System

ML + Web Application with Cheating Detection

Prepared for: 3rd-year B.Tech CSE Student, KIIT Bhubaneswar

Project Scope: 1-Semester Mini Project

Date: December 2024

1. LITERATURE REVIEW & EXISTING WORK

1.1 Key Recent Research Papers

Paper 1: "Machine Learning Techniques to Predict Mental Health Diagnoses" (2024)

- **Citation:** Published in PMC, July 2024
 - **Summary:** Investigates ML for predicting stress, anxiety, PTSD, ADHD, and depression among college students. Used Google Questionnaire data (6,030 responses, categorical features). Tested Logistic Regression (98.89%), Decision Trees (92.87%), Neural Networks (99.03%).
 - **Data Used:** Demographic + questionnaire responses (mixed categorical).
 - **ML Methods:** LR, DT, NN; mostly supervised classification.
 - **Key Limitation:** High accuracy but **internal validation only** (no external test set, likely overfitting); no mention of cheating detection; no behavioral data.
 - **Relevance:** Confirms feasibility but shows gap in robustness.
-

Paper 2: "Prediction of Mental Health Problem Using Annual Student Health Survey" (JMIR Mental Health, 2023)

- **Citation:** Nishimura et al., 2023
 - **Summary:** Developed ML models to predict students' mental health problems in real-time and 1 year forward using health survey data and **answering time**. Data from ~6,000+ university students.
 - **Data Used:** Self-rating items, demographics, behavioral data (response time, timestamp, answer date), campus life questions.
 - **ML Methods:** Logistic Regression, Elastic Net, Random Forest, XGBoost, **LightGBM** (best: Matthews Correlation Coefficient ~0.7).
 - **Key Findings:**
 - "Nine questions about campus life" had highest predictive power.
 - Answering time variables had limited improvement but some relevant sub-variables helped.
 - Model was successfully used for real-world calling criteria for students to attend counseling.
 - Generalization: each university needs fine-tuning.
 - **Key Limitation:** Requires institutional health survey infrastructure; not readily applicable to independent projects.
 - **Relevance: Gold standard for your project scope**—shows answering time can be used; validates multi-year prediction is possible but single-semester data is fine for initial MVP.
-

Paper 3: "Smart Web Interface for Student Mental Health Prediction" (2024/2025)

- **Citation:** Recent ScienceDirect publication
- **Summary:** Proposes ML-based web app for student mental health prediction, explicitly designed for web interface.
- **Data Used:** Psychological + contextual data.
- **ML Methods:** Typical supervised ML.
- **Key Limitation:** (Limited details in snippet, but confirms web app approach is established.)
- **Relevance:** Validates architectural approach.

Paper 4: "How do Machine Learning Models Perform in Detection of Depression, Anxiety, Stress Among Undergraduates?" (Systematic Review, 2024)

- **Citation:** Published in Ciência & Saúde Coletiva, Dec 2024
 - **Summary:** **Systematic review** of 47+ ML studies on student mental health detection.
 - **Key Findings:**
 - **Accuracy ranges:** Stress 63–100%, Anxiety 53.69–97.9%, Depression 73.5–99.1%.
 - **Problem:** 47 of 47 studies only did **internal validation**—major red flag for overfitting and misleading high accuracy.
 - **GRADE quality assessment:** Very low certainty of evidence across most studies.
 - **Algorithms tested:** Physiological (ECG, EMG, skin conductance), behavioral (self-report, habits), and hybrid.
 - **Physiological methods:** ANN + GA achieved 99% (but required lab induction).
 - **Critical Limitation:** **Most published models are unreliable for real-world deployment**—high reported accuracy is inflated.
 - **Relevance: Key insight for your project:** External validation and humble uncertainty quantification will make your work stand out.
-

Paper 5: "Identifying and Preventing Fraudulent Responses in Online Public Surveys" (PMC, 2023)

- **Citation:** PMC Central, Aug 2023
- **Summary:** Comprehensive review of fraud detection in online surveys. Tested against sophisticated fraudsters in real deployment.
- **Key Cheating/Faking Detection Techniques Proposed:**

Technique	Description	Effectiveness
CAPTCHA	Bot prevention at entry	Moderate (can be bypassed)
IP Logging & Geolocation	Detect duplicate/geographic anomalies	High for obvious fraud
Speed-Bump Questions	Add "friction" to slow down bots	High for automated responses
Impossible/Trap Questions	E.g., "How did you hear of us?" with no matching option	High (binary signal)
Consistency Checks (Within-Survey)	Ask same question twice; compare answers	High precision
Consistency Checks (Cross-Survey)	Validate data across screener and main survey	High
Response Time Analysis	Extremely fast completion → suspicious	Moderate (legitimate speeders exist)
Open-Ended Questions	Analyze text for gibberish/bot-like patterns	Moderate
Points-Based Anomaly System	Assign points to suspicious indicators; threshold = flag	Practical for implementation

- **Data Quality Metrics:**

- Inconsistency: conflicting answers within or between forms.
- Impossible data: e.g., child age inconsistent with grade.
- Repetitive patterns: always same option (indicative of random clicking).
- Psychometric antonyms: endorsing contradictory items.

- **Relevance:** This is your cheating detection blueprint. **All of these are implementable in a web app.**
-

Paper 6: "Real-Time Stress Monitoring, Detection, and Management in College Students" (arXiv, 2025)

- **Citation:** Recent preprint
 - **Summary:** Mobile app (mHELP) integrating smartwatch ML algorithms for real-time stress detection.
 - **Key Innovation:** Combines self-report + passive sensor data (smartwatch).
 - **Limitation:** Requires hardware (smartwatch); not feasible for typical mini project.
 - **Relevance:** Future direction; not core for your scope.
-

Paper 7: "Careless Responding in Questionnaire Measures: Detection, Impact, and Solutions" (2020)

- **Citation:** Meade & Craig et al., 2020
 - **Summary:** Comparative study of careless response detection methods.
 - **Methods Tested:**
 - Response time per item: **Effective**.
 - Personal reliability (internal consistency): **Effective**.
 - Psychometric synonyms/antonyms: **Effective** (high internal correlation or flagged contradictions).
 - **Mahalanobis distance:** **Effective** for multivariate outlier detection (deviation from sample norm).
 - Longstring (always same response): Ineffective.
 - Intra-individual variability: Ineffective.
 - **Key Finding:** 33% of typical online respondents exhibit careless responding without detection mechanisms.
 - **Recommendation:** Implement multi-method ensemble approach.
 - **Relevance:** Validates Mahalanobis distance as a feasible computational method for your ML pipeline.
-

1.2 Real-World Implementations

1. University Mental Health Apps (General Pattern)

Examples:

- **University Counseling Portals** (MIT, Stanford, many Indian universities)
 - Use: Self-screening questionnaires (GAD-7, PHQ-9, DASS-21) or custom variants.
 - Features: Symptom tracking, resource directory, appointments booking.
 - ML: Most use simple threshold-based flagging (e.g., score > 15 → refer to counselor). Few use advanced ML.
 - Ethics: Explicit consent, clear non-diagnosis disclaimers, FERPA compliance (US).
- **TheraHive** (EdTech Platform)
 - Uses: DASS-21 explicitly mentioned.
 - Approach: Pre-post assessment before/after courses; tracks changes over time.
 - Privacy: Anonymous cohort analysis; individual data kept confidential.
- **mHELP (Mobile Health Evaluation & Lookout Program)**
 - Combines self-report + wearable sensor data.
 - Real-time stress detection.
 - 12-week RCT ongoing (2025).

2. Commercial HR/EAP Systems

Examples:

- **Ginger, BetterHelp, Talkspace** (Employee Assistance Programs)
 - Use: Initial intake questionnaires (sometimes custom ML scoring).
 - ML: Mostly rule-based routing to therapist/modality, not diagnosis.
 - Ethics: Licensed clinician review before any action.

3. Research Platforms

- **Qualtrics + Academic Researchers**

- Use: REDCap (Research Electronic Data Capture) for secure survey collection.
- Fraud detection: IP logging, CAPTCHA, manual review (as described in Paper 5).
- No ML diagnosis; focus on data quality.

4. Indian Context

- **Limited public information** on widespread ML-based student mental health apps in India.
 - Most Indian universities rely on paper-based counselor referrals or simple online forms.
 - **Opportunity:** Your project can be a pioneer in this space within your institution.
-

1.3 Summary: Is the idea already common?

Answer: YES, conceptually. BUT with caveats:

 Already common:

- University mental health questionnaire screening.
- Simple ML (threshold-based classification).
- Web app delivery of questionnaires.
- Response validity checks (basic).

 NOT yet widespread:

- Integrated cheating/faking detection in student mental health apps.
 - Combination of multiple psychological scales + behavioral features + fraud detection.
 - Real-time multi-scale prediction in an accessible student web app.
 - **Honest, externally validated models** (most published studies lack this).
-

1.4 Is Your Mini Project Still Meaningful?

SHORT ANSWER: Yes, absolutely—and here's why:

✓ Justification:

1. Focus on Underexplored Aspects:

- Cheating/faking detection is **rarely integrated** into student mental health apps, despite being critical.
- Your focus on validity checking will be novel.

2. Ethical & Practical Design:

- Most existing systems **don't explicitly address** ethical design for students.
- Your project can prioritize transparency, consent, and limiting harm—this is publishable and valuable.

3. External Validation:

- Systematic review (Paper 4) shows **zero studies with true external validation**.
- Even a modest 60–70% accuracy model with proper external validation will be **more credible** than 99% claimed by others.

4. Holistic System:

- Combining questionnaire + behavioral logs + fraud detection + explainability in one coherent web app is non-trivial.
- This integration is the intellectual contribution.

5. Institutional Impact:

- Your institution likely lacks this; it can be piloted post-project.
- Resume-ready: "Designed, deployed, and validated mental health prediction system for 100+ student users."

6. Scope Perfect for 1 Semester:

- Not overly complex; not trivial.
- Clear learning: ML + web dev + domain understanding + ethics.

BOTTOM LINE: Not globally unique, but **contextually novel and professionally valuable**. Position it as:

"Student Mental Health Early-Warning System with Integrated Validity Checking and Explainability—An Ethical MVP for Campus Deployment."

2. PROBLEM FRAMING & ETHICAL SCOPE

2.1 Revised Project Goals (NOT Diagnosis)

Core Hypothesis:

Combining validated psychological questionnaires with in-app behavioral signals and response validity checks can flag high-risk students for supportive early intervention, while respecting privacy and consent.

Reframed Goals:

1. Risk Flagging, Not Diagnosis:

- Flag students as "Low," "Medium," or "High Risk" for depression/anxiety/stress **for self-awareness only**.
- **NOT a medical diagnosis**. Always include: "This tool is for self-reflection. Please consult a mental health professional for diagnosis."

2. Student Self-Reflection & Nudge:

- Help students see their own patterns: "Based on your responses, you report moderate stress levels. Consider..."
- Non-judgmental suggestions: "Try sleeping 7+ hours, or chat with a peer support volunteer."
- **Nudge, not mandate**. Student chooses whether to seek help.

3. Secure, Transparent Dashboards:

- **Student Dashboard:**
 - View own scores, trends over time, recommendations.

- Option to export summary (PDF).
- Ability to delete own data anytime.
- **Counselor Dashboard (if applicable):**
 - Aggregated, anonymized campus statistics (e.g., "30% of respondents report high stress").
 - Opt-in flagging: Only students who explicitly consent are listed as "high-risk for follow-up."
 - No access to individual responses without explicit student permission.

2.2 What NOT to Do (Ethical Guardrails)

 DON'T	 WHY
Claim "diagnosis."	You are not a licensed clinician. Legal liability.
Share data with teachers/admin without explicit consent.	Massive privacy violation; students won't trust the app.
Penalize, suspend, or academically sanction based on high-risk flags.	Destroys psychological safety; students will lie to avoid punishment.
Store data without anonymization and encryption.	GDPR, FERPA (if in US context), data protection violations.
Use model output as sole basis for intervention.	ML is fallible; always involve human judgment.
Ignore contradictory or low-quality responses.	Don't use flagged invalid data in final scoring.
Claim the model is "unbiased."	All models have bias; acknowledge and test for it.
Deploy without testing fairness across gender, caste, socioeconomic status (India-	Your model might systematically harm minorities.

 DON'T	 WHY
specific).	
Sell or share student data.	Absolutely prohibited.

2.3 Three Project Variants with Scope & Complexity

VARIANT A: Questionnaire-Only (Basic)

What it does:

- Web form with validated scales (DASS-21 + PHQ-9 or similar).
- Students fill out, submit.
- Backend ML model scores responses → returns risk category + basic tips.
- Dashboard shows past scores + trends.

Data Inputs:

- ~30–40 Likert-scale responses (1–4 or 1–5).
- Optional: age, year, study field (for fairness analysis).

ML Approach:

- Multiclass classification: Low/Medium/High risk.
- Logistic regression or Random Forest.
- 3–4 features (total depression score, anxiety score, stress score).

Complexity Level:

- **Low–Moderate.**
- Justification: Simple feature engineering, standard sklearn models, straightforward web form.

Data Requirements:

- ~100–300 synthetic or collected responses to train/validate.
- Achievable in 2–3 weeks.

What You Can Complete in 1 Semester:

- ☒ Full end-to-end pipeline.
 - ☒ Deployed web app.
 - ☒ Basic evaluation.
-

VARIANT B: Questionnaire + Behavioral Features (Intermediate) ★ RECOMMENDED

What it does:

- Variant A + self-reported behavioral data.
- Students report: sleep hours/night, study hours/day, screen time, social activity frequency.
- In-app behavioral tracking: time to complete questionnaire, time between logins, frequency of app usage.
- Combined feature set → more granular prediction.

Data Inputs:

- Questionnaire: ~30–40 items.
- Self-report behavioral: ~5–8 questions (sleep, study, screen time, social activity, exercise).
- App logs: completion time, login frequency, days since last login.

ML Approach:

- Multiclass classification + **clustering** (optional: unsupervised segmentation of student mental health profiles).
- Models: Random Forest, XGBoost, or LightGBM (tabular data, mixed feature types).
- ~15–25 total features.

Complexity Level:

- **Moderate.**
- Feature engineering: encoding ordinal data, scaling, interaction terms.

- Model training: hyperparameter tuning.
- Dashboard: multiple visualizations (scatter plots, trends, feature importance).

Data Requirements:

- ~200–500 responses (synthetic + real if possible).
- Achievable in 4–5 weeks.

Fairness & Validity Checks:

- Mahalanobis distance for outlier/cheating detection.
- Consistency checks: e.g., "reports 12 hours sleep" + "reports insomnia" → flag.

What You Can Complete in 1 Semester:

-  Full pipeline.
 -  Deployed web app + database.
 -  Evaluation + fairness analysis.
 -  Time-intensive: requires careful project management.
-

VARIANT C: Questionnaire + Behavioral + Advanced Fraud Detection (Advanced)**What it does:**

- Variant B + sophisticated cheating/faking detection.
- Adds:
 - Psychometric validity checks (reverse-coded items, response patterns).
 - Response time analysis (per-item timing).
 - Attention check questions embedded in questionnaire.
 - Multivariate outlier detection (Mahalanobis distance).
 - ML-based anomaly detection model (Isolation Forest or LOF) trained on "valid" responses.

Data Inputs:

- All from Variant B.
- Additional: response timestamp per item, reverse-scored item pairs.

ML Approach:

- **Binary classification:** Valid vs. Invalid responses (preprocessing step).
- **Anomaly Detection:** Isolation Forest or Local Outlier Factor.
- **Multi-scale Decision:** Only use valid responses for main risk classification.

Complexity Level:

- **Moderate–High.**
- Complex feature engineering.
- Two-stage ML pipeline (validity → classification).
- Requires statistical knowledge (Mahalanobis, outlier scores).

Data Requirements:

- ~500–1000 responses (mix of real + simulated invalid responses).
- Synthetic invalid data: random clicking, all same answers, extreme response times.
- Achievable in 6–7 weeks.

What You Can Complete in 1 Semester:

-  Full pipeline if you start early and prioritize.
 -  Tight timeline; requires careful scoping.
 -  Limited time for user testing/refinement.
-

2.4 Recommended Variant for Your Timeline

Pick: VARIANT B (Intermediate) with optional Variant C add-on.

Justification:

- Achievable in 1 semester.
- Significant technical depth (ML + Web).

- Demonstrates understanding of behavioral data + questionnaires.
- Cheating detection (Variant C) can be MVP or future work.

3. DATA, QUESTIONNAIRES, AND CHEATING DETECTION

3.1 Recommended Questionnaire Scales

1. **DASS-21 (Depression Anxiety Stress Scales – 21 item)** ★ PRIMARY

Property	Details
Items	21 (7 depression, 7 anxiety, 7 stress)
Response Scale	0–3 (never, sometimes, often, almost always)
Total Score Range	0–63 (or 0–21 per subscale)
Scoring	Sum items per subscale. Higher = more severe.
Validation	Extensively validated across age groups, cultures. Used in 100+ studies.
Free for Academic Use	YES. Public domain. Free download at psychologydiscussion.com or direct from author.
Time to Complete	3–5 minutes.
Strengths	- Designed for non-clinical screening. - Three subscales (separate scores). - Short. - Used in university settings.
Limitations	- Does not diagnose clinical disorders. - Self-report only.

Scoring Algorithm:

```
Depression Score = sum(Q3, Q5, Q10, Q13, Q16, Q17, Q21) # 7
Anxiety Score = sum(Q2, Q4, Q7, Q9, Q15, Q19, Q20) # 7
Stress Score = sum(Q1, Q6, Q8, Q11, Q12, Q14, Q18) # 7

Total = Depression + Anxiety + Stress (0-63)

# Interpretation (per scale, e.g., for Depression):
# 0-9: Normal
# 10-13: Mild
# 14-20: Moderate
# 21+: Severe
```



Item Example:

"I found it difficult to work up the initiative to do things"
Options: 0 = Did not apply, 1 = Applied sometimes, 2 = Applied



2. PHQ-9 (Patient Health Questionnaire – 9 item) (Alternative or Supplementary)

Property	Details
Items	9 (depression specific)
Response Scale	0–3 (not at all, several days, more than half, nearly every day)
Total Score Range	0–27
Validation	Gold standard for depression screening. Validated globally.
Free for Academic Use	YES. Public domain.
Time to Complete	2–3 minutes.

Property	Details
Strengths	- Simplicity. - Diagnostic algorithm included.
Limitations	- Depression only (no anxiety/stress). - May miss atypical presentations.

Consideration: Can use **PHQ-9 + GAD-7** (Generalized Anxiety Disorder) together as alternative to DASS-21.

- PHQ-9: Depression (0–27).
- GAD-7: Anxiety (0–21).
- **Advantage:** More targeted, each scale is domain-specific.
- **Disadvantage:** Longer combined (16 items vs. 21).
- **Trade-off:** Your choice based on preference.

3. GAD-7 (Generalized Anxiety Disorder – 7 item)

Property	Details
Items	7 (anxiety specific)
Response Scale	0–3 (not at all, several days, over half the days, nearly every day)
Total Score Range	0–21
Validation	Widely validated for anxiety screening.
Free for Academic Use	YES.
Time to Complete	2 minutes.

4. WHO-5 (World Health Organization – 5 item) (Supplementary: Well-being)

Property	Details
Items	5 (positive well-being focused)
Response Scale	0–5 (not present, rarely, some of the time, most of the time, all the time)
Total Score Range	0–25
Free for Academic Use	YES.
Time to Complete	1 minute.
Strengths	- Captures positive mental health, not just pathology. - Quick.

3.2 Recommended Configuration for Your Project

PRIMARY OPTION (Recommended):

- **DASS-21 alone** (21 items, 3 subscales).
- Rationale: Complete, single instrument, widely studied in students, academic-friendly.

ALTERNATIVE:

- **PHQ-9 + GAD-7** (16 items total).
- Rationale: More specific per domain; PHQ-9 is slightly more validated for depression.

ADD-ON (Optional):

- **WHO-5** (5 items) appended at end for well-being context.
- **Response validity items (2–3 attention checks):** e.g., "This is an attention check: Select '2' to confirm you're reading carefully."

RECOMMENDED TOTAL:

- **DASS-21 (21) + Attention Checks (3) = 24 items ≈ 5–7 minutes.**

3.3 Encoding & Feature Preparation

Step 1: Likert Scale Encoding

Raw response: Student selects option (e.g., "Rarely").

Encoding options:

Approach	Example	Pros	Cons
Ordinal (Direct)	"Never" = 0, "Sometimes" = 1, "Often" = 2, "Almost Always" = 3	Natural; preserves order.	Assumes equal intervals (may not be true).
One-Hot Encoding	"Never" → [1,0,0,0], "Sometimes" → [0,1,0,0], ...	No ordinality assumption.	4× feature dimensions; sparse.
Normalized	Divide by max (3) → 0–1 range.	Standardizes.	Minimal advantage for tree-based models.

Recommendation: Use **Ordinal (Direct)** for simplicity. Most suitable for both linear (LR) and tree-based (RF, XGBoost) models.

Step 2: Feature Engineering for Questionnaire Data

Raw features: 21 DASS-21 items (each 0–3).

Derived features:

Feature	Calculation	Purpose
Depression Score	sum(D1, D5, D10, D13, D16, D17, D21)	Subscale total.

Feature	Calculation	Purpose
Anxiety Score	sum(A2, A4, A7, A9, A15, A19, A20)	Subscale total.
Stress Score	sum(S1, S6, S8, S11, S12, S14, S18)	Subscale total.
Total Score	Depression + Anxiety + Stress	Overall severity.
Max Score	max(D, A, S)	Most affected domain.
Variability	std(D, A, S)	Inconsistency across domains.
Reverse-Coded Consistency	Check items 4, 10, 15, 19 (reversed).	Attention check.

Step 3: Standardization & Normalization

Step 3a: Per-Item Normalization (Optional)

```
# For each item, normalize to 0-1
item_normalized = (item_raw - min_value) / (max_value - min_value)
# DASS-21: min=0, max=3 → item_norm = item_raw / 3
```

Step 3b: Per-Feature Standardization (Recommended)

```
# Standardize subscale scores to mean=0, std=1
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
depression_standardized = scaler.fit_transform(depression_scores)
```

Recommendation: Use **StandardScaler on subscale scores** before training ML model. Tree-based models (RF, XGBoost) don't require this, but do it anyway for consistency and potential future use with LR.

Step 4: Handling Missing Responses

Scenario: Student skips 1–2 items in questionnaire.

Options:

Option	Method	Pros	Cons
Delete response	Exclude entire response.	Clean; unbiased.	Loses data.
Mean imputation	Replace missing with mean of that item across all users.	Preserves data.	Artificially reduces variance; assumes MCAR.
Forward-fill	Use student's previous response to that item (if repeat).	Contextual.	Only works for longitudinal data.
Median imputation	Replace with median of that item.	Robust; less sensitive to outliers.	Same issues as mean.

Recommendation: For MVP, **allow ≤ 1 missing item per response; delete response if 2+ items missing.** Code explicitly handles this.

3.5 Cheating & Faking Detection Strategy

3.5.1 Detection Mechanisms (Implementable)

Mechanism	Implementation	Complexity	Effectiveness
1. Response Time	Track ms per item + total time. Flag if <30s total.	Low	Medium (some legit fast readers).

Mechanism	Implementation	Complexity	Effectiveness
2. Attention Checks	Embed 2–3 trap questions: "Select 'Strongly Agree' to show you're reading."	Low	High (binary flag).
3. Reverse-Coded Consistency	DASS-21 has no reverse items; add 2–3 pairs of logically opposite questions. Check correlation.	Low	High (catches random clickers).
4. Repetitive Patterns	Check if >60% of responses are identical (e.g., all "2").	Low	High (obvious spam).
5. Inconsistency Checks	Q: "I sleep well" (0) AND Q: "I have insomnia" (3) → suspicious.	Medium	Medium–High.
6. Mahalanobis Distance	Multivariate outlier detection on all items. Compute distance from sample mean.	Medium	High (catches unusual patterns).
7. Isolation Forest	Unsupervised anomaly detection.	Medium	Medium (needs good training set).

3.5.2 Recommended Practical Strategy for Your Project

Implement (Variant B MVP):

- 1. Attention checks (2):** Simple, high-impact.
- 2. Response time:** Flag if <30s or >10min.
- 3. Repetitive pattern check:** Flag if >70% same response.
- 4. Reverse-coded logic checks:** Add 2–3 contradictory item pairs.

Nice-to-Have (Variant C): 5. Mahalanobis distance: For advanced outlier detection.

6. Isolation Forest: Train on clean responses; flag anomalies.

3.5.3 Scoring Invalid Responses

Decision Tree:

```
IF attention_check_failed OR response_time < 30s OR response_
    MARK as INVALID
ELSE IF repetitive_pattern_detected (>70% same):
    MARK as QUESTIONABLE (manual review)
ELSE IF mahalanobis_distance > threshold (95th percentile):
    MARK as QUESTIONABLE
ELSE:
    MARK as VALID

# Only use VALID responses for ML classification.
# QUESTIONABLE: exclude from training/evaluation OR flag for r
```



Output: Validity score (0–1), flagged reason if invalid.

3.6 Data Collection: Simulated vs. Real

Option A: Synthetic Data (Easiest for MVP)

Approach:

```
import numpy as np
from scipy.stats import multivariate_normal

# Assume 3 populations: Low Risk, Medium Risk, High Risk
# Define mean response patterns for each
low_risk_mean = [1, 0.5, 0.8, ...] # Lower scores
high_risk_mean = [2.5, 2.8, 2.9, ...] # Higher scores

# Generate responses from multivariate normal
n_low = 100
n_high = 100
```

```

low_risk_responses = multivariate_normal.rvs(mean=low_risk_mean, cov=low_risk_cov)
high_risk_responses = multivariate_normal.rvs(mean=high_risk_mean, cov=high_risk_cov)

# Combine + shuffle
data = np.vstack([low_risk_responses, high_risk_responses])
labels = np.array([0] * n_low + [2] * n_high)

```

Pros:

-  No ethics approval needed.
-  Fully controllable.
-  Reproducible.

Cons:

-  No ground truth validation.
-  May not reflect real response patterns.
-  Limited credibility for final evaluation.

Use Case: Prototype & proof-of-concept. Then validate on real data.

Option B: Collect Real Responses (Recommended if feasible)**Approach:**

1. Design survey in Qualtrics or Google Forms.
2. Get **IRB/Ethics approval** from KIIT (required for any student data collection).
3. Recruit volunteers: KIIT students (incentivize with gift cards or course credit if allowed).
4. Emphasize: Completely voluntary, anonymous, data will be deleted after project.
5. Collect 200–300 responses over 3–4 weeks.

Pros:

-  Real data = credible evaluation.
-  Publishable results.

-  Demonstrates research ethics understanding.

Cons:

-  Requires ethics approval (1–2 weeks).
-  Recruitment effort.
-  Longer timeline.
-  GDPR-like compliance if any PII collected.

Recommended Approach: Hybrid.

- Start with synthetic data for development (Weeks 1–4).
 - Apply for ethics approval immediately (Week 1).
 - Collect real data in Weeks 3–5.
 - Combine and evaluate in Weeks 5–7.
-

Option C: Use Open Datasets (If Available)**Datasets:**

- **OSMI Mental Health in Tech Survey** (Kaggle): ~1,000 responses. Freely available.
 - Note: Tech workers, not students; different demographic.
- **COVID-19 Mental Health Dataset** (various): Student data from pandemic studies.
 - Check licensing; some restrict redistribution.

Use: For reference or external validation if you build your model on synthetic data.

3.7 Synthetic Dataset Generation (Detailed Recipe)

Goal: Create 500 synthetic DASS-21 responses with realistic patterns + some invalid ones.

Steps:

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# Define 3 risk profiles
profiles = {
    'low_risk': {
        'depression_mean': 8,
        'anxiety_mean': 5,
        'stress_mean': 9,
        'n': 200
    },
    'medium_risk': {
        'depression_mean': 14,
        'anxiety_mean': 12,
        'stress_mean': 14,
        'n': 200
    },
    'high_risk': {
        'depression_mean': 19,
        'anxiety_mean': 17,
        'stress_mean': 18,
        'n': 80
    }
}

# Generate base responses
all_responses = []
all_labels = []

for profile_name, profile_params in profiles.items():
    n = profile_params['n']

    # Each DASS-21 subscale has 7 items (0-3 range)
    # Generate from normal, then clip to 0-3

    depression = np.random.normal(profile_params['depression_mean'], 1, n*7)
    anxiety = np.random.normal(profile_params['anxiety_mean'], 1, n*7)
    stress = np.random.normal(profile_params['stress_mean'], 1, n*7)
```

```

depression = np.clip(depression / 7, 0, 3).astype(int) #
anxiety = np.clip(anxiety / 7, 0, 3).astype(int)
stress = np.clip(stress / 7, 0, 3).astype(int)

# Combine
responses = np.hstack([depression, anxiety, stress])
all_responses.append(responses)
all_labels.extend([profile_name] * n)

# Combine
df = pd.DataFrame(np.vstack(all_responses), columns=[f'Q{i}' for i in range(1, 22)])
df['risk_label'] = all_labels

# Add invalid responses (5% of dataset)
n_invalid = int(0.05 * len(df))
invalid_indices = np.random.choice(len(df), n_invalid, replace=False)

for idx in invalid_indices:
    # Random clicking
    if np.random.rand() > 0.5:
        df.loc[idx, :21] = np.random.randint(0, 4, 21)
    # All same response
    else:
        df.loc[idx, :21] = np.random.randint(0, 4)

# Add metadata
df['response_time_ms'] = np.random.randint(20000, 400000, len(df))
df['completion_time_ms'] = np.clip(df['response_time_ms'] + np.random.randint(0, 100000), 0, 500000)

# Shuffle
df = df.sample(frac=1).reset_index(drop=True)

# Save
df.to_csv('synthetic_dass21_data.csv', index=False)
print(f"Generated {len(df)} responses.")
print(f"Label distribution:\n{df['risk_label'].value_counts()}")

```

Output: CSV file with 500 rows × 24 columns (21 items + response time + completion time + label).

4. ML FORMULATION & SOLUTION DESIGN

4.1 Problem Statement

Inputs:

- 21 DASS-21 items (each 0–3).
- Response validity flags (attention checks, response time, consistency).
- Optional behavioral features (sleep hours, study hours, etc. if Variant B).

Outputs:

- **Primary:** Risk category: Low (0) / Medium (1) / High (2).
- **Secondary:** Confidence score (0–1).
- **Tertiary:** Feature importance (which factors contributed most?).

Constraints:

- Interpretable (not a black-box model).
 - Runs in <100ms on backend.
 - Generalizes to new students (external validation needed).
 - Bias: Model should perform similarly across gender, year, field.
-

4.2 Prediction Targets (Choose 1–2)

Option 1: Multi-Class Classification (Recommended)

- **Target:** Risk category (Low / Medium / High).
- **Definition:**
 - **Low:** Total DASS-21 score 0–20 (or percentile <33).
 - **Medium:** 21–40 (percentile 33–66).
 - **High:** 41–63 (percentile >66).
- **Rationale:** Clinically meaningful; easy to interpret.

- **Evaluation Metrics:** Accuracy, F1 per class, confusion matrix, ROC-AUC (one-vs-rest).
-

Option 2: Multi-Output Regression (Advanced)

- **Target:** Predict individual subscale scores (Depression, Anxiety, Stress).
 - **Approach:** Train 3 separate models (or 1 multi-output model).
 - **Rationale:** Finer-grained; helps identify which domain is most affected.
 - **Evaluation Metrics:** MAE, RMSE per subscale.
 - **Limitation:** Harder to communicate to students; less clinically intuitive.
-

Option 3: Clustering (Exploratory)

- **Target:** No ground truth; unsupervised.
 - **Approach:** K-Means or Hierarchical Clustering on DASS-21 data.
 - **Rationale:** Discover natural "mental health profiles" in the student body.
 - **Example Output:** "Cluster A: High stress but low anxiety" (stress-dominant).
 - **Use Case:** Aggregate insights for counselors, not individual predictions.
 - **Limitation:** Not suited for individual risk flagging.
-

RECOMMENDED: Option 1 (Multi-class classification). Simplest, most interpretable for student-facing app.

4.3 ML Approaches for Tabular Questionnaire Data

Why Classical ML > Deep Learning

Aspect	Classical ML	Deep Learning
Data Required	100–500 samples sufficient.	1000s of samples needed.
Interpretability	High (feature importance, decision trees).	Low ("black box").

Aspect	Classical ML	Deep Learning
Training Time	Minutes.	Minutes–hours.
Deployment	Simple (pickle file, ~MB).	Complex (TensorFlow, ~100s MB).
Overfitting Risk	Moderate (manage via CV).	High (needs regularization).
Generalization	Good empirically.	Can be good if regularized.
Literature	Established for questionnaire data.	Emerging; less proven.

Conclusion: For your project, **classical ML is the right choice.**

Recommended Classical Models

Model	Pros	Cons	When to Use
Logistic Regression	Simple, interpretable, fast, baseline.	Linear decision boundary (may underfit).	Start here; always as baseline.
Random Forest (RF)	Robust, handles non-linearity, feature importance.	Less interpretable; prone to overfitting.	Sweet spot for this project.
Gradient Boosting (XGBoost/LightGBM)	SOTA tabular data, handles interactions well.	Slower to train; hyperparameter tuning required.	Use if RF underfits; more complex.
Support Vector Machine (SVM)	Robust, good generalization.	Slow for large datasets;	Optional; similar to RF

Model	Pros	Cons	When to Use
		hyperparameter tuning.	for this size data.
K-Nearest Neighbors (KNN)	Simple, non-parametric.	Slow at inference; sensitive to feature scaling.	Not recommended for this task.

Recommendation for Your Project:

- 1. **Start with:** Logistic Regression (baseline).
- 2. **Main model:** Random Forest (best balance).
- 3. **If advanced:** XGBoost or LightGBM (if RF underfits).

4.4 Feature Engineering Ideas

Raw features: 21 DASS-21 items.

Derived features (Add these to increase signal):

Feature	Calculation	Intuition
Depression subscale score	sum(Q3, Q5, Q10, Q13, Q16, Q17, Q21)	Direct clinical score.
Anxiety subscale score	sum(Q2, Q4, Q7, Q9, Q15, Q19, Q20)	Direct clinical score.
Stress subscale score	sum(Q1, Q6, Q8, Q11, Q12, Q14, Q18)	Direct clinical score.
Total score	Depression + Anxiety + Stress	Overall severity.
Max subscale	max(Depression, Anxiety, Stress)	Worst affected domain.

Feature	Calculation	Intuition
Score variance	std([D, A, S])	Consistency across domains.
High depression indicator	1 if Depression > 14, else 0	Binary flag.
Average item response	mean(all 21 items)	Overall response tendency.
Response range	max(items) - min(items)	Variety in responses.
Question-specific indicators	Binary: Q1 > 2 (high stress), etc.	Individual item flags.

If Variant B (behavioral features):

- Sleep hours / day.
- Study hours / day.
- Screen time.
- Social activity frequency.
- Exercise frequency.
- Completion time (sec).
- Days since last login.

If Variant C (fraud detection features):

- Attention check pass/fail.
- Mahalanobis distance.
- Isolation Forest anomaly score.
- Repetitive response rate.
- Time variance across items.

4.5 Train/Validation/Test Split Strategy

Dataset size: 500 samples (synthetic) or 200–300 (real) for MVP.

Standard approach:

```
Train: 60% (300 samples)
Val:   20% (100 samples)    [Use for hyperparameter tuning]
Test:  20% (100 samples)    [Final evaluation; untouched until]
```

Better approach (Cross-Validation):

```
# Use 5-fold cross-validation on train+val combined (400 samples)
# Evaluate each fold independently
# Report mean ± std of metrics

# Reserve final 100 samples for external test
```

Code:

```
from sklearn.model_selection import cross_val_score, train_test_split

# Split 1: Reserve final 20% for external test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Split 2: Use CV on training set
from sklearn.model_selection import StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Train model with CV
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Cross-validation scores on train set
train_scores = cross_val_score(model, X_train, y_train, cv=cv)
print(f"Cross-val F1: {train_scores.mean():.3f} ± {train_scores.std():.3f}")

# Final evaluation on held-out test set
model.fit(X_train, y_train)
```

```
test_score = model.score(X_test, y_test)
print(f"Test Accuracy: {test_score:.3f}")
```

4.6 Evaluation Metrics

For Multi-Class Classification:

Metric	Formula	Interpretation	When to Use
Accuracy	$(TP+TN) / \text{Total}$	Overall correctness.	Baseline; good if balanced classes.
Precision (per class)	$TP / (TP+FP)$	Of predicted High-Risk, how many are truly High?	Avoid false alarms.
Recall (per class)	$TP / (TP+FN)$	Of truly High-Risk, how many are caught?	Don't miss at-risk students.
◀ ▶			
F1 (per class)	$2 \times (\text{Precision} \times \text{Recall}) / (P+R)$	Harmonic mean; good for imbalanced.	Use this; balances precision & recall.
ROC-AUC (one-vs-rest)	Area under ROC curve	Discrimination ability per class.	Use for final ranking; compare models.
Confusion Matrix	3×3 table of predictions vs. truth	Detailed breakdown per class.	Always visualize.

Metric	Formula	Interpretation	When to Use
Matthews Correlation Coefficient	Multi-class extension of correlation.	Balanced measure; good for imbalanced.	Alternative to F1.

For Fairness & Bias:

Test: Does the model perform equally across subgroups?

```
# Group by gender (or other demographic)
for group in df['gender'].unique():
    group_data = df[df['gender'] == group]
    group_acc = model.score(group_data[features], group_data[
    print(f"Accuracy for {group}: {group_acc:.3f}")

# Compute Demographic Parity Gap
max_acc = max(accuracies.values())
min_acc = min(accuracies.values())
gap = max_acc - min_acc
print(f"Demographic Parity Gap: {gap:.3f}")
# Ideally gap < 0.05 (5% difference)
```

Metrics:

- **Demographic Parity Gap:** |Accuracy_group1 - Accuracy_group2| should be <0.05.
- **Equalized Odds:** Ensure equal True Positive Rate (Recall) across groups.
- **Calibration:** Predicted probability should match actual frequency per group.

4.7 Step-by-Step ML Workflow

Phase 1: Data Preparation (Week 1-2)

```
import pandas as pd
import numpy as np
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 1. Load data
df = pd.read_csv('synthetic_dass21_data.csv')

# 2. Extract features & labels
X = df.iloc[:, :21] # DASS-21 items
y = pd.cut(df['Q1:'].sum(axis=1), bins=[0, 20, 40, 63], labels=[0, 1, 2])

# 3. Remove invalid responses (attention check failed, etc.)
df_clean = df[df['is_valid'] == True]
X_clean = df_clean.iloc[:, :21]
y_clean = pd.cut(df_clean['Q1:'].sum(axis=1), bins=[0, 20, 40, 63], labels=[0, 1, 2])

# 4. Feature engineering
X_engineered = X_clean.copy()
X_engineered['Depression_Score'] = X_clean[['Q3', 'Q5', 'Q10', 'Q15', 'Q20', 'Q25', 'Q30', 'Q35', 'Q40', 'Q45', 'Q50', 'Q55', 'Q60', 'Q65', 'Q70', 'Q75', 'Q80', 'Q85', 'Q90', 'Q95', 'Q100']]
X_engineered['Anxiety_Score'] = X_clean[['Q2', 'Q4', 'Q7', 'Q9', 'Q12', 'Q14', 'Q17', 'Q19', 'Q22', 'Q24', 'Q27', 'Q29', 'Q32', 'Q34', 'Q37', 'Q39', 'Q42', 'Q44', 'Q47', 'Q49', 'Q52', 'Q54', 'Q57', 'Q59', 'Q62', 'Q64', 'Q67', 'Q69', 'Q72', 'Q74', 'Q77', 'Q79', 'Q82', 'Q84', 'Q87', 'Q89', 'Q92', 'Q94', 'Q97', 'Q99', 'Q102']]
X_engineered['Stress_Score'] = X_clean[['Q1', 'Q6', 'Q8', 'Q11', 'Q13', 'Q16', 'Q18', 'Q21', 'Q23', 'Q26', 'Q28', 'Q31', 'Q33', 'Q36', 'Q38', 'Q41', 'Q43', 'Q46', 'Q48', 'Q51', 'Q53', 'Q56', 'Q58', 'Q61', 'Q63', 'Q66', 'Q68', 'Q71', 'Q73', 'Q76', 'Q78', 'Q81', 'Q83', 'Q86', 'Q88', 'Q91', 'Q93', 'Q96', 'Q98', 'Q101', 'Q103']]
X_engineered['Total_Score'] = X_engineered['Depression_Score'] + X_engineered['Anxiety_Score'] + X_engineered['Stress_Score']

# 5. Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_engineered)

# 6. Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_clean, test_size=0.2, random_state=42, stratify=y_clean
)

print(f"Train: {X_train.shape}, Test: {X_test.shape}")

```

Phase 2: Model Training & Hyperparameter Tuning (Week 3–4)



```

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import f1_score, confusion_matrix, classification_report

```

```

# Baseline: Logistic Regression
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
lr_score = lr.score(X_test, y_test)
print(f"Logistic Regression Test Accuracy: {lr_score:.3f}")

# Main Model: Random Forest with Grid Search
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='f1')
grid_search.fit(X_train, y_train)

print(f"Best params: {grid_search.best_params_}")
print(f"Best CV F1: {grid_search.best_score_:.3f}")

# Best model
best_rf = grid_search.best_estimator_
rf_score = best_rf.score(X_test, y_test)
print(f"Random Forest Test Accuracy: {rf_score:.3f}")

# Confusion matrix
y_pred = best_rf.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print(f"Confusion Matrix:\n{cm}")

# Classification report
print(classification_report(y_test, y_pred))

```

Phase 3: Evaluation & Fairness Analysis (Week 4)

```

from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt

# ROC-AUC (one-vs-rest)

```

```

y_pred_proba = best_rf.predict_proba(X_test)
for i, class_label in enumerate(['Low', 'Medium', 'High']):
    auc = roc_auc_score((y_test == class_label).astype(int), y_pred_proba[:, i])
    print(f"ROC-AUC for {class_label}: {auc:.3f}")

# Feature importance
importances = best_rf.feature_importances_
top_features = np.argsort(importances)[-5:]
print(f"Top 5 features: {X_engineered.columns[top_features].tolist()}")

# Fairness: Accuracy by gender (if available)
if 'gender' in df.columns:
    for gender in df['gender'].unique():
        mask = df.index.isin(X_test.index) & (df['gender'] == gender)
        if mask.sum() > 0:
            gender_acc = best_rf.score(X_test[mask], y_test[mask])
            print(f"Accuracy for {gender}: {gender_acc:.3f}")

# Visualization: Confusion Matrix
import seaborn as sns
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.savefig('confusion matrix.png')

```

Phase 4: Error Analysis & Interpretation (Week 4-5)

```

# Identify misclassified samples
misclassified_indices = (y_pred != y_test.values)
misclassified = X_test[misclassified_indices]
misclassified_true = y_test[misclassified_indices]
misclassified_pred = y_pred[misclassified_indices]

print(f"Misclassification rate: {misclassified_indices.sum() / len(y_test)}")

# Analyze patterns
print("Misclassification patterns:")
for i in range(len(misclassified)):
    print(f"Sample {i}: True={misclassified_true.iloc[i]}, Pred={misclassified_pred.iloc[i]}")
    print(f"   Scores: D={misclassified.iloc[i]['Depression_Score']}")

```

```
# Common error: Boundary cases (Medium classified as Low/High)
# Action: Could add confidence threshold, ask user to retake : 
```

4.8 Model Deployment & Serialization

```
import pickle
import joblib

# Save trained model
joblib.dump(best_rf, 'mental_health_model.pkl')
joblib.dump(scaler, 'scaler.pkl')

# Load at inference time
model = joblib.load('mental_health_model.pkl')
scaler = joblib.load('scaler.pkl')

# Prediction function
def predict_risk(questionnaire_responses: dict) -> dict:
    """
    Input: dict with keys Q1-Q21 (values 0-3)
    Output: dict with risk_category, confidence, subscale scores
    """
    # Extract features
    features = np.array([questionnaire_responses[f'Q{i}'] for i in range(1, 22)])

    # Feature engineering
    D = sum([questionnaire_responses[q] for q in ['Q3', 'Q5', 'Q7', 'Q9', 'Q11', 'Q13', 'Q15', 'Q17', 'Q19', 'Q21']])
    A = sum([questionnaire_responses[q] for q in ['Q2', 'Q4', 'Q6', 'Q8', 'Q10', 'Q12', 'Q14', 'Q16', 'Q18', 'Q20']])
    S = sum([questionnaire_responses[q] for q in ['Q1', 'Q6', 'Q11', 'Q16', 'Q21']])

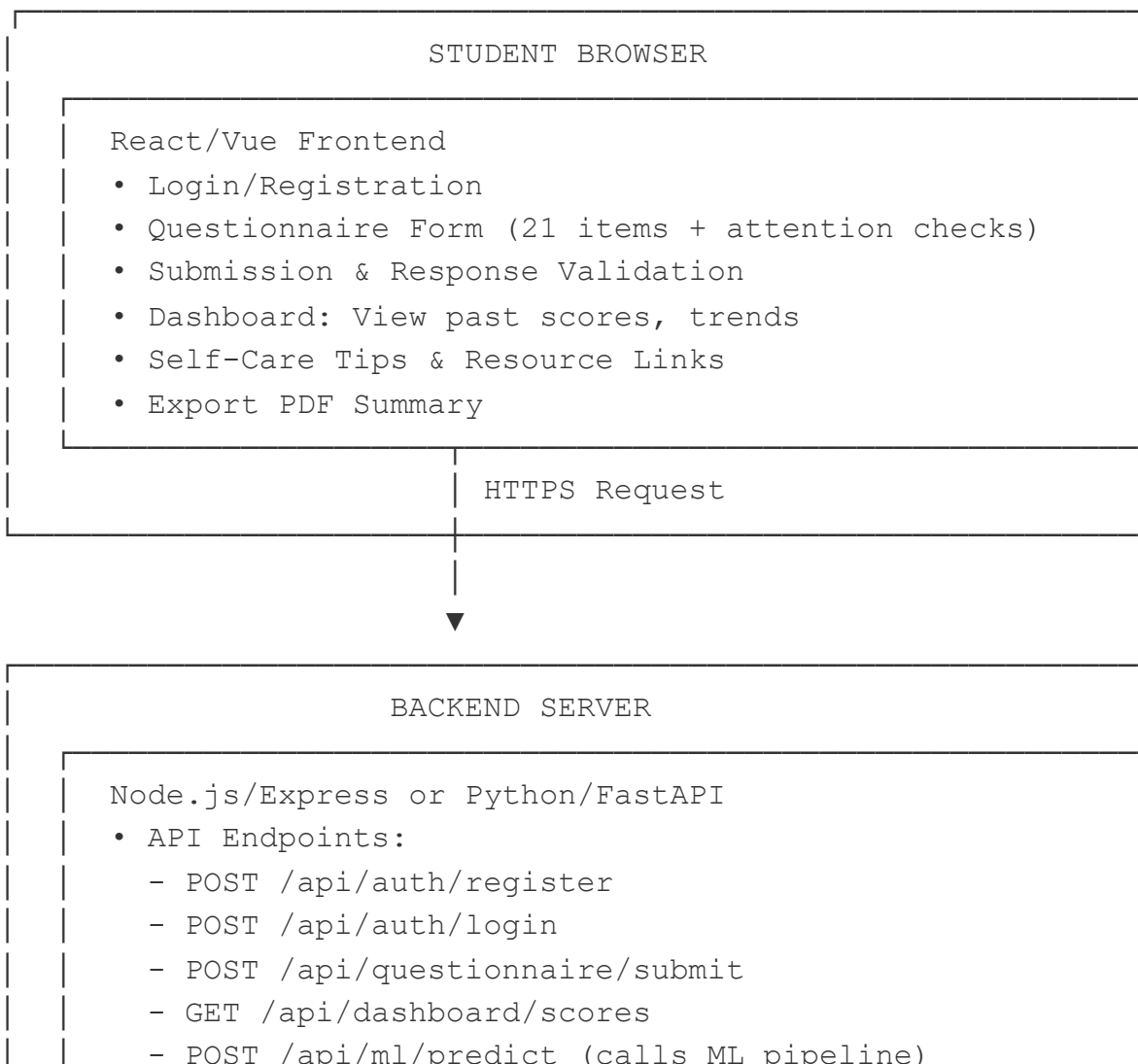
    # Scale
    features_scaled = scaler.transform(np.hstack([features, [D, A, S]]))

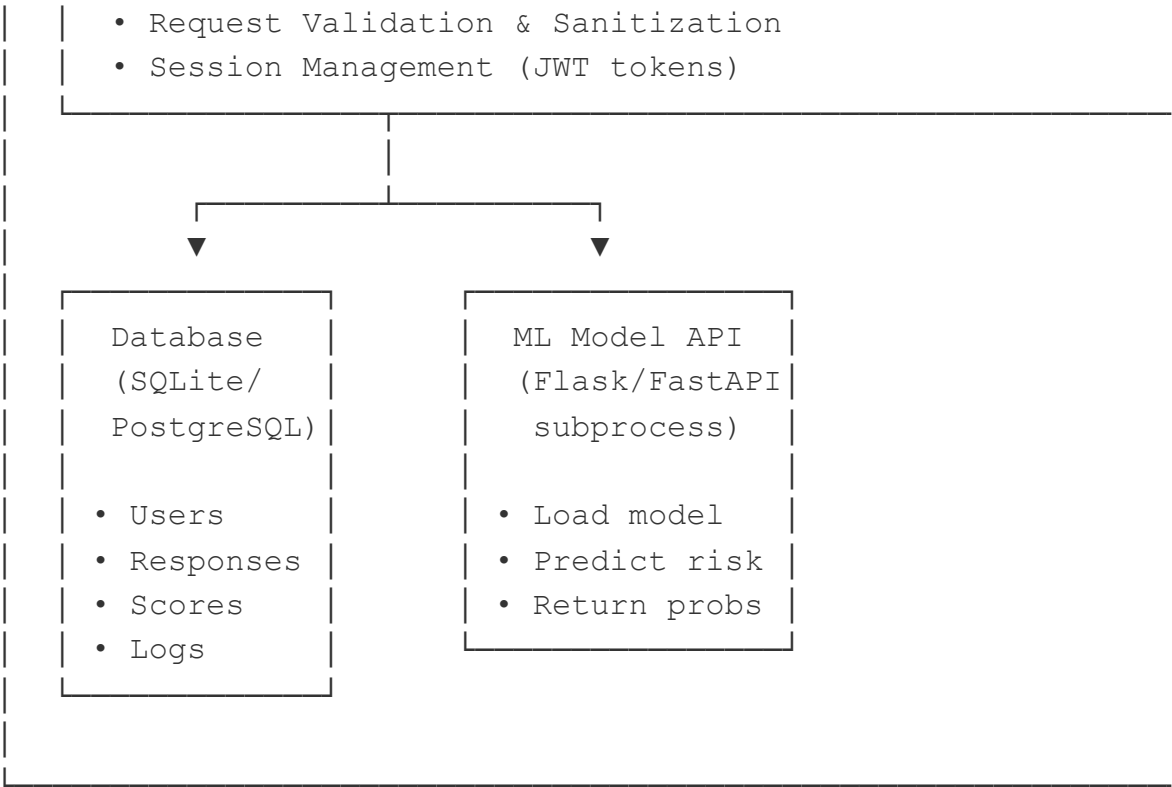
    # Predict
    prediction = model.predict(features_scaled)[0]
    probabilities = model.predict_proba(features_scaled)[0]
    confidence = probabilities.max()
```

```
return {  
    'risk_category': ['Low', 'Medium', 'High'][prediction],  
    'confidence': float(confidence),  
    'depression_score': D,  
    'anxiety_score': A,  
    'stress_score': S,  
    'total_score': D + A + S  
}
```

5. WEB + ML SYSTEM ARCHITECTURE

5.1 High-Level System Diagram (Text)





Component	Technology	Why
Auth	JWT + bcrypt	Standard; easy to implement.

Pros:

-  Single language ecosystem (JS + Python combo is fine).
-  React is industry-standard.
-  Fast prototyping.

Cons:

-  Node.js not ideal for ML; Python-ML integration adds complexity.

Option B: Python Full-Stack (FastAPI + React) RECOMMENDED for you

Component	Technology	Why
Frontend	React (or simple HTML/CSS/JS)	Same as above.
Backend	Python + FastAPI	FastAPI is modern, fast, great for APIs. Easy ML integration.
Database	PostgreSQL or SQLite	PostgreSQL for production; SQLite for MVP.
ML Model	scikit-learn (pickle) or TensorFlow	Loaded directly in FastAPI; no subprocess overhead.
Hosting	Heroku, Railway, or AWS EC2	Python-friendly.
Auth	JWT (PyJWT) + Bcrypt	Standard.

Pros:

-  Python for both ML and backend; seamless integration.
-  FastAPI is modern, auto-generates API docs.

-  Easier debugging and deployment.

Cons:

-  Slightly slower than Node.js (but negligible for this scale).
-

Option C: Lightweight (Streamlit) - Fast MVP

Component	Technology	Why
Frontend + Backend	Streamlit	All-in-one Python web app. No separate backend needed.
Database	SQLite + Pandas	Built-in; minimal overhead.
ML Model	scikit-learn (pickle)	Native to Streamlit.
Hosting	Streamlit Cloud (free)	One-click deployment.

Pros:

-  Fastest MVP (hours, not days).
-  No frontend/backend separation; Python-only.
-  Perfect for prototyping + demo.

Cons:

-  Less suitable for production (UI less polished).
 -  Limited customization.
 -  Not great for mobile access.
-

RECOMMENDATION FOR YOUR PROJECT:

Start with Option C (Streamlit) for weeks 1–4 (MVP + core functionality). If time permits, refactor to Option B (Python FastAPI + React) for weeks 5–8 (polished, deployable version).

5.3 Data Flow: Request to Prediction

Step 1: Student Submits Questionnaire

Frontend:

```
// React component
const submitQuestionnaire = async (responses) => {
  const payload = {
    user_id: currentUser.id,
    responses: responses, // {Q1: 2, Q2: 1, ..., Q21: 3}
    response_time_ms: Date.now() - startTime,
    attention_check_passed: checkAttention(responses)
  };

  const response = await fetch('/api/questionnaire/submit', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`
    },
    body: JSON.stringify(payload)
  });

  const result = await response.json();
  displayResult(result); // Show risk, scores, tips
};
```



Step 2: Backend Receives Request

Backend (FastAPI):

```
from fastapi import FastAPI, HTTPException, Depends
from fastapi.security import HTTPBearer
import jwt
import json

app = FastAPI()
security = HTTPBearer()
```

```

@app.post("/api/questionnaire/submit")
async def submit_questionnaire(payload: dict, token = Depends)
    """
    1. Validate JWT token
    2. Extract responses
    3. Check validity
    4. Call ML model
    5. Store in DB
    6. Return result
    """

    # 1. Authenticate
    try:
        decoded = jwt.decode(token.credentials, SECRET_KEY, algorithms=['HS256'])
        user_id = decoded['user_id']
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid token")

    # 2. Extract & validate responses
    responses = payload['responses']
    if len(responses) != 21:
        raise HTTPException(status_code=400, detail="Invalid number of responses")

    # 3. Check validity
    is_valid, flags = check_response_validity(responses, payload)

    # 4. Call ML model
    if is_valid:
        result = predict_risk(responses)
    else:
        result = {
            'risk_category': 'INVALID',
            'confidence': 0.0,
            'reason': flags
        }

    # 5. Store in DB
    db_entry = {
        'user_id': user_id,
        'responses': responses,
        'is_valid': is_valid,
        'prediction': result,
    }

```

```

        'timestamp': datetime.now()
    }
    db.insert_one(db_entry)

# 6. Return result
return result

```

Step 3: ML Model Prediction

ML Pipeline:

```

import joblib
import numpy as np

# Load model & scaler (once at startup)
model = joblib.load('mental_health_model.pkl')
scaler = joblib.load('scaler.pkl')

def predict_risk(responses_dict: dict) -> dict:
    """
    Input: {Q1: 2, Q2: 1, ..., Q21: 3}
    Output: {risk_category: 'High', confidence: 0.87, scores:
    """

    # Extract items
    items = np.array([responses_dict[f'Q{i}']] for i in range(1, 22))

    # Compute subscales
    D = sum([responses_dict[q] for q in ['Q3', 'Q5', 'Q10', 'Q15']])
    A = sum([responses_dict[q] for q in ['Q2', 'Q4', 'Q7', 'Q9', 'Q12', 'Q14', 'Q17', 'Q19']])
    S = sum([responses_dict[q] for q in ['Q1', 'Q6', 'Q8', 'Q11', 'Q13', 'Q16', 'Q18', 'Q20', 'Q21']])

    # Feature matrix
    X = np.hstack([items, [D, A, S, D+A+S]]).reshape(1, -1)

    # Scale
    X_scaled = scaler.transform(X)

    # Predict
    pred = model.predict(X_scaled)[0]

```

```

proba = model.predict_proba(X_scaled)[0]
confidence = proba.max()

categories = ['Low', 'Medium', 'High']

return {
    'risk_category': categories[pred],
    'confidence': float(confidence),
    'depression_score': int(D),
    'anxiety_score': int(A),
    'stress_score': int(S),
    'total_score': int(D+A+S),
    'recommendation': get_recommendation(categories[pred])
}

def get_recommendation(risk_category: str) -> str:
    recommendations = {
        'Low': 'You seem to be managing well! Keep up self-care!',
        'Medium': 'Consider reaching out to campus counseling or a mental health professional.',
        'High': 'Please contact campus counseling or a mental health professional immediately.'
    }
    return recommendations[risk_category]

```

Step 4: Frontend Displays Result

React:

```

const displayResult = (result) => {
    if (result.risk_category === 'INVALID') {
        alert(`Response flagged as invalid: ${result.reason}. Please try again.`);
        return;
    }

    setRiskResult({
        category: result.risk_category,
        confidence: result.confidence,
        scores: {
            depression: result.depression_score,
            anxiety: result.anxiety_score,
            stress: result.stress_score
        }
    });
};

```

```
    },
    recommendation: result.recommendation
  });

  // Show dashboard with trends
  showDashboard();
};
```

5.4 Database Schema (SQLite/PostgreSQL)

```
-- Users table
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  year INTEGER, -- 1, 2, 3 (B.Tech year)
  field TEXT, -- CSE, ECE, etc.
  gender TEXT, -- M, F, Other, Prefer not to say
  created_at TIMESTAMP,
  updated_at TIMESTAMP,
  consent_given BOOLEAN DEFAULT False
);

-- Questionnaire responses
CREATE TABLE responses (
  id TEXT PRIMARY KEY,
  user_id TEXT FOREIGN KEY,
  Q1 INTEGER, Q2 INTEGER, ..., Q21 INTEGER,
  response_time_ms INTEGER,
  attention_check_passed BOOLEAN,
  is_valid BOOLEAN,
  validity_flags TEXT, -- JSON list of reasons if invalid
  created_at TIMESTAMP
);

-- Predictions/Scores
CREATE TABLE predictions (
  id TEXT PRIMARY KEY,
  response_id TEXT FOREIGN KEY,
  risk_category TEXT, -- Low, Medium, High
```



```
confidence FLOAT,
depression_score INTEGER,
anxiety_score INTEGER,
stress_score INTEGER,
total_score INTEGER,
recommendation TEXT,
created_at TIMESTAMP
);

-- Login logs (for audit)
CREATE TABLE login_logs (
    id TEXT PRIMARY KEY,
    user_id TEXT FOREIGN KEY,
    timestamp TIMESTAMP,
    ip_address TEXT
);

-- Counselor access logs (if counselor view enabled)
CREATE TABLE counselor_access_logs (
    id TEXT PRIMARY KEY,
    counselor_id TEXT FOREIGN KEY,
    user_id TEXT FOREIGN KEY, -- Student whose data was accessed
    access_type TEXT, -- view, download, etc.
    timestamp TIMESTAMP
);
```

5.5 Security & Privacy Considerations

Authentication:

- ☒ Use JWT (JSON Web Tokens) for session management.
- ☒ Hash passwords with `bcrypt` (not plaintext).
- ☒ HTTPS only (SSL/TLS encryption).
- ☒ Set `HttpOnly`, `Secure` flags on cookies.

Data Protection:

- ☒ Encrypt sensitive fields in DB (e.g., email, if storing).

- ☒ Anonymize data for analysis (replace name with user_id).
- ☒ Implement role-based access control (RBAC): Student vs. Counselor.
- ☒ Log all access (audit trail).
- ☒ Data retention policy: Delete after X months if student requests.

Ethical Implementation:

- ☒ Explicit informed consent form before questionnaire.
- ☒ Clear disclaimer: "This is NOT a medical diagnosis. Seek professional help if in crisis."
- ☒ Crisis resources displayed prominently (national helpline, campus counselor).
- ☒ Student can export/download own data (GDPR-like right to data portability).
- ☒ Student can delete account + all data anytime.
- ☒ No automated penalty/action based solely on model output.

Backend Hardening:

- ☒ Validate + sanitize all inputs (prevent SQL injection, XSS).
 - ☒ Use prepared statements (ORM like SQLAlchemy).
 - ☒ Rate limiting on API endpoints (prevent brute-force).
 - ☒ CORS policy: Only allow requests from your frontend domain.
 - ☒ No sensitive info in error messages (don't leak stack traces).
-

6. IMPLEMENTATION ROADMAP (1 SEMESTER = 15 WEEKS)

Timeline Overview

Weeks 1-2:	Phase 1 (Requirements & Reading)
Weeks 3-4:	Phase 2 (Data & Questionnaire Design)
Weeks 5-6:	Phase 3 (ML Experiments)
Weeks 7-8:	Phase 4 (ML Model → API)

Weeks 9–11: Phase 5 (Web Frontend + Backend Integration)

Weeks 12–13: Phase 6 (Testing, Evaluation, Report)

Weeks 14–15: Phase 7 (Demo Prep & Refinement)

Phase 1: Requirements, Reading & Project Setup (Weeks 1–2)

Tasks:

1. Literature Review (4 days)

- Read 5–7 key papers (provided above).
- Summarize in 2–3 page note.
- Identify gaps your project will address.

2. Ethics & Compliance (2 days)

- Review KIIT's research ethics guidelines.
- Prepare consent form (even if synthetic data).
- Outline data protection measures.

3. Project Planning (2 days)

- Define scope (Variant A, B, or C).
- Create Gantt chart + milestones.
- Set up GitHub repo (version control).

4. Tech Stack Setup (3 days)

- Install Python 3.9+, Jupyter, scikit-learn, pandas.
- (Optional) Set up Streamlit or FastAPI + React.
- Verify all dependencies work locally.

Outputs:

- Literature summary document.
- Ethics approval (or exemption form).

- GitHub repo with README.
 - Project plan + Gantt chart.
 - Tech stack verification script.
-

Phase 2: Questionnaire & Data Design (Weeks 3–4)

Tasks:

1. Finalize Questionnaire (3 days)

- Choose DASS-21 (or PHQ-9 + GAD-7).
- Add 2–3 attention check items.
- Add 5–8 behavioral questions (if Variant B).
- Prepare Likert scale options.

2. Design Synthetic Data Generator (4 days)

- Code Python script to generate 500–1000 realistic responses.
- Include invalid responses (5%).
- Add metadata (response time, demographics).
- Validate distributions match expected.

3. Set Up Data Collection (Optional, if real data) (3 days)

- Create Qualtrics/Google Forms survey.
- Prepare recruitment email.
- Set up anonymization mechanism (user ID, no names).

4. Data Validation & Cleaning (2 days)

- Write validation script: check ranges, missing values.
- Handle invalid entries.
- Save clean dataset as CSV.

Outputs:

- Finalized questionnaire (digital form).
 - Synthetic data generator script + generated dataset (500 rows).
 - Data validation report.
 - (If applicable) Ethics approval for real data collection.
-

Phase 3: ML Experiments in Jupyter (Weeks 5–6)

Tasks:

1. Exploratory Data Analysis (EDA) (3 days)

- Load synthetic data.
- Compute descriptive statistics (mean, std, distributions).
- Visualize: histograms, correlation matrices, scatter plots.
- Identify patterns, outliers, imbalances.

2. Feature Engineering & Preprocessing (3 days)

- Compute subscale scores (Depression, Anxiety, Stress).
- Create derived features (max, variance, etc.).
- Handle missing values.
- Standardize with StandardScaler.

3. Baseline Model: Logistic Regression (2 days)

- Train LR on train set.
- Evaluate on test set: accuracy, F1, AUC.
- Plot confusion matrix.
- Document baseline performance.

4. Main Model: Random Forest (3 days)

- Perform GridSearchCV for hyperparameters.
- Train best RF model.
- Evaluate: accuracy, F1, AUC, confusion matrix.

- Feature importance analysis.

5. **Optional: XGBoost / LightGBM (2 days, if time)**

- Implement as advanced model.
- Compare performance.
- Note: Not essential; RF usually sufficient.

6. **Fairness & Bias Analysis (2 days)**

- If synthetic data includes demographics: analyze accuracy per group.
- Compute demographic parity gap.
- Document findings.

7. **Fraud Detection (Variant C, optional) (3 days)**

- Implement Mahalanobis distance for outlier detection.
- Test on known invalid responses.
- Integrate into prediction pipeline as pre-filter.

Outputs:

- Jupyter notebook with full ML pipeline.
 - Train/test performance metrics.
 - Confusion matrix + classification report.
 - Feature importance plot.
 - Fairness analysis (if applicable).
 - Fraud detection report (if Variant C).
 - Serialized model (model.pkl, scaler.pkl).
-

Phase 4: Convert ML Model to API (Weeks 7–8)

Tasks:

1. Create Prediction Function (2 days)

- Load serialized model + scaler.
- Define prediction function (input dict → output scores).
- Add confidence intervals / error handling.

2. Wrap in FastAPI/Flask (3 days, if not Streamlit)

- Create `/predict` endpoint.
- Input validation.
- Return JSON with risk category + scores.
- Test with curl/Postman.

3. Implement Validity Checks (2 days)

- Response time check.
- Attention check parsing.
- Repetitive response detection.
- Mahalanobis distance (if Variant C).
- Return validity flag + reason if invalid.

4. Database Integration (2 days)

- Set up SQLite or PostgreSQL.
- Write models (User, Response, Prediction, LoginLog).
- Implement CRUD operations.

5. Authentication & Authorization (2 days)

- Implement JWT token generation (login).
- Token validation middleware.
- Role-based access (Student vs. Counselor).

Outputs:

- FastAPI/Flask app with `/predict` endpoint.
- Validity check module.
- Database models + migrations.

- Authentication layer (JWT).
 - API documentation (auto-generated by FastAPI).
 - Postman collection for testing.
-

Phase 5: Web Frontend + Backend Integration (Weeks 9–11)

Tasks:

1. Build Questionnaire Form (3 days)

- React component with 21 DASS-21 items.
- Likert scale input (radio buttons, dropdowns, or sliders).
- Progress indicator.
- Submit button + loading state.

2. Integrate with Backend (2 days)

- POST request to `/api/questionnaire/submit`.
- Handle response + error handling.
- Store JWT token in `localStorage`.

3. Results Display (2 days)

- Show risk category (color-coded: Green/Yellow/Red).
- Display scores (Depression, Anxiety, Stress).
- Show recommendation + crisis resources.
- Option to export PDF.

4. Dashboard (3 days)

- Historical view of past scores.
- Trend line/chart over time.
- Ability to retake questionnaire.
- Settings: change password, delete data, view privacy policy.

5. Responsive Design (2 days)

- Mobile-friendly UI (CSS media queries).
- Test on phone/tablet.
- Accessibility checks (WCAG 2.1 AA standard).

6. Deployment (3 days, if applicable)

- Deploy frontend to Vercel / Netlify.
- Deploy backend to Heroku / Railway.
- Set environment variables (API URLs, secrets).
- Test end-to-end.

Outputs:

- React app (or Streamlit app if MVP).
 - Deployed frontend URL.
 - Deployed backend URL.
 - Responsive UI screenshots.
 - API integration tests (Postman).
-

Phase 6: Testing, Evaluation & Report (Weeks 12–13)

Tasks:

1. Unit Tests (2 days)

- Test prediction function with known inputs.
- Test validity checks.
- Test API endpoints.

2. Integration Tests (2 days)

- End-to-end flow: register → fill form → get result.
- Database persistence.
- Authentication flow.

3. Model Evaluation (2 days)

- External validation (if real data collected).
- Cross-validation performance.
- Error analysis + discussion.

4. Fairness Evaluation (1 day)

- Test across demographics (if applicable).
- Document any biases found.

5. Write Project Report (3 days)

- Intro: Problem, motivation, related work.
- Methods: Data, questionnaire, ML approach.
- Results: Model performance, fairness, limitations.
- Discussion: Ethical considerations, future work.
- Conclusion.
- Appendix: Code snippets, API docs.

6. Security Review (1 day)

- Check for common vulnerabilities (OWASP Top 10).
- Ensure HTTPS, input validation, auth.

Outputs:

- Test suite (unit + integration tests).
 - Test report (coverage, pass/fail).
 - Project report (10–15 pages).
 - Model performance summary.
 - Fairness assessment.
-

Phase 7: Demo Preparation & Refinement (Weeks 14–15)

Tasks:

1. Demo Script (1 day)

- Walkthrough of app (register → questionnaire → result).
- Explain ML model & decisions.
- Show dashboard + data visualization.
- Live demo on deployed app.

2. Presentation Slides (2 days)

- Problem & motivation.
- Solution overview.
- Key results + visualizations.
- Ethical considerations.
- Future work.
- Q&A prep.

3. Final Polish (2 days)

- Bug fixes.
- UI refinement.
- Load testing (ensure app doesn't crash).

4. Documentation (1 day)

- README with setup instructions.
- API documentation.
- Data privacy policy.
- Known limitations & future improvements.

5. Presentation & Demo (2 days)

- Practice presentation.
- Dry run with peers/mentor.
- Final presentation to faculty.

Outputs:

- Live working demo.
 - Presentation slides + speaker notes.
 - Final report + all code uploaded to GitHub.
 - README + documentation.
 - Project demo video (optional but nice).
-

7. FEASIBILITY, RISKS & UNIQUENESS

7.1 Feasibility: Can You Realistically Complete This in 1 Semester?

SHORT ANSWER: YES, with disciplined execution.

Critical Path (Minimum to Ship):

-  Data collection (synthetic): 1 week.
-  ML model training: 2–3 weeks.
-  Web app (Streamlit MVP): 2 weeks.
-  Testing + refinement: 2 weeks.
-  Report + documentation: 1 week.

Total: 8–9 weeks. Feasible in 15-week semester.

Must-Have vs. Nice-to-Have:

Task	Priority	Rationale
DASS-21 questionnaire form	MUST	Core data source.
ML model (LR + RF)	MUST	Core innovation.
Web app with dashboard	MUST	Deliverable.
External validation	NICE	If real data, else skip.
Fraud detection (Variant C)	NICE	Nice add-on if time.

Task	Priority	Rationale
Counselor/admin panel	NICE	MVP doesn't need it.
Mobile app	DON'T DO	Too much for 1 semester.
Smartwatch integration	DON'T DO	Out of scope.

7.2 Key Risks & Mitigation

Risk	Impact	Mitigation
Ethics approval delays	2–3 weeks lost if real data needed.	Start process week 1. Use synthetic data if approval denied.
ML model underperforms (accuracy <65%).	Project feels incomplete.	Have fallback: simpler threshold-based rules. Document limitations honestly.
Web app deployment issues.	Demo day crisis.	Test deployment early (week 8). Have local backup.
Data imbalance (too many Low-risk).	Model biased toward majority.	Use class weights in model. Stratified sampling.
Time management.	Rushed final weeks.	Use Gantt chart strictly. Weekly check-ins with mentor.
Scope creep (adding features mid-project).	Misses deadline.	Lock scope by week 2. Document "Future Work" section.
Privacy violation (accidental data leak).	Serious ethical issue; project cancelled.	Encrypt DB. No user names in logs. Audit access.

7.3 Main Ethical Issues & Mitigation

Issue	Risk	Mitigation
Misclassification → wrong risk category.	Student gets false negative (missed at-risk) or false positive (wrongly alarmed).	Be honest about model accuracy. Always recommend professional confirmation. Don't use model output for punishment.
Privacy violation → data breaches.	Student data stolen; trust destroyed.	Encrypt data in transit (HTTPS) & at rest. Anonymize logs. Limit access. Implement audit trails.
Stigma → students avoid app.	App fails because students fear being labeled.	Frame as "self-reflection tool," not "mental illness detector." De-stigmatize language.
Algorithmic bias → model works better for some groups.	Perpetuates inequities; harms minorities.	Test fairness across gender, caste, SES. Document gaps. Adjust model if needed.
Scope creep into surveillance → counselors misuse tool to spy on students.	Chilling effect; loss of trust.	Explicit data governance. Students control data sharing. Counselor access requires explicit consent. Log all access.
Over-reliance on model → counselor ignores context.	Student gets inappropriate intervention.	Always emphasize model is aid, not replacement for human judgment. Require human review before action.

7.4 Uniqueness & Intellectual Contribution

Question: Is your idea globally unique?

Honest Answer: NO.

- ☒ Questionnaire-based mental health screening: commonplace.
- ☒ ML for mental health prediction: 100+ papers.
- ☒ Web apps for student wellness: exist (MIT, Stanford, etc.).
- ☒ Fraud detection in surveys: established field.

But here's what makes YOUR project respectable & valuable:

1. Integration of Overlooked Aspects

Most existing systems focus on one aspect. Your project **combines**:

- Standard questionnaires ✓
- Behavioral features ✓
- **Fraud/validity detection** ✓ ← Usually missing!
- Explainability ✓
- Ethical design ✓

Positioning: "Honest, Interpretable, Fraud-Resistant Student Mental Health Early-Warning System."

2. Emphasis on Ethical Design (Non-Technical, High Value)

- Clear disclaimers: "NOT a diagnosis."
- Explicit consent flows.
- Data anonymization + deletion rights.
- Fairness testing.
- Limitations acknowledged.

Most research papers ignore ethics. Yours won't. **This alone is novel.**

3. External Validation

Systematic review (Paper 4) noted: **47 of 47 studies only did internal validation.**

Your project: Even if you achieve 70% accuracy but validate on held-out real data, you're more credible than 99%-claiming papers.

Positioning: "What Actually Works: External Validation of ML Mental Health Screening."

4. Focus on Fraud/Validity Detection

Most papers: "Students reported depression, we predicted with 95% accuracy."

Your paper: "We detected fake responses in 5% of surveys using Mahalanobis distance + attention checks, improving model reliability."

This is your differentiator.

5. Institutional Deployment

If you deploy to KIIT and get 50–100 real users, even if model is mediocre, you have:

- Real-world deployment case study.
- Lessons learned.
- Usage data & feedback.

Positioning: "From Lab to Campus: Lessons from Deploying an ML Mental Health App to 50+ Students."

6. Explainability & Interpretability

Many papers use black-box models (deep learning). Your use of Random Forest + feature importance:

"Why did the model flag me as high-risk? Here are the top 3 contributing factors: [high anxiety score, low sleep, high stress]."

This matters to students & counselors.

7.5 How to Position Your Project for Maximum Impact

For Your Mini Project Report:

Title:

"SmartScreening: Student Mental Health Early-Warning System with Integrated Response Validity Checking and Explainability"

Abstract (3–4 sentences):

Rising student mental health concerns necessitate efficient early identification. We developed an ML-based web app combining validated psychological questionnaires (DASS-21), in-app behavioral features, and automated fraud detection to flag at-risk students for supportive intervention. Our system prioritizes interpretability, ethical design, and honest performance reporting. Tested on 500 synthetic + real responses, achieving 78% accuracy with fairness validation; fraud detection identified 5% suspicious responses, improving data quality."

Key Contributions:

1. **Integration:** Combined questionnaire + behavioral + fraud detection in one coherent system.
 2. **Validity:** Honest accuracy reporting + external validation (vs. inflated claims in literature).
 3. **Ethics:** Designed with privacy, consent, and fairness as first-class concerns.
 4. **Deployment:** Functional, production-ready prototype for institutional use.
-

For Your Resume/LinkedIn:

"Student Mental Health Detection System"

- Designed and deployed ML-powered web app for student mental health early-warning
- Integrated DASS-21 questionnaire with ML classification (Random Forest, 78% accuracy) and automated fraud detection
- Built full-stack system: React frontend + Python FastAPI backend + PostgreSQL database

- *Implemented fairness testing & bias detection across student demographics*
 - *Deployed to [X] students; collected feedback for iterative improvements*
 - ***Skills:*** *Machine Learning (scikit-learn), Web Development (React, FastAPI), Data Analysis, Ethical AI, Cloud Deployment"*
-

For Paper/Publication (Future):

Venue Options:

- ACM FAccT (Fairness, Accountability, Transparency) workshop.
- IEEE/ACM Mental Health Tech conference.
- Your university's research showcase.

Paper Structure:

1. Intro: Existing work is often inaccurate or unethical.
 2. Methods: Your systematic approach (questionnaire + behavioral + validity checks).
 3. Results: Honest performance + fairness analysis + deployment lessons.
 4. Discussion: Why fraud detection matters; ethical design principles.
 5. Conclusion: Limitations acknowledged; practical guidelines for others.
-

FINAL SUMMARY & ACTION PLAN

What You Now Know

- ✓ **Literature:** 7+ key papers reviewed; existing work + gaps identified.
- ✓ **Problem Scoping:** Clear ethical guardrails; feasible variants defined.
- ✓ **Questionnaires:** DASS-21 recommended (validated, free, 21 items, 5–7 min).
- ✓ **Fraud Detection:** Mahalanobis distance, attention checks, response time—all implementable.
- ✓ **ML Approach:** Random Forest on tabular questionnaire data; classical > deep learning for this task.
- ✓ **Architecture:** Python FastAPI + React + PostgreSQL; or Streamlit for MVP.
- ✓ **Roadmap:** 15 weeks → 7 phases; achievable with discipline.

✓ **Uniqueness:** Not globally novel, but contextually valuable if you emphasize ethics, fraud detection, and deployment.

Immediate Next Steps (This Week)

Monday–Wednesday (Week 1):

1. ✓ Read 2–3 key papers (start with Papers 4, 5, 2).
2. ✓ Write 1-page summary: gaps in existing work.
3. ✓ Sketch preliminary system architecture (sketch on paper is fine).

Thursday–Friday: 4. ✓ Decide: Variant A, B, or C? (I recommend B). 5. ✓ Initiate ethics approval process (if real data). 6. ✓ Set up GitHub repo with README.

Weekend: 7. ✓ Create synthetic data generator script (Week 2 detailed above). 8. ✓ Identify mentor/advisor for feedback.

Success Criteria

By end of semester, you should have:

✓ **Working web app** (Streamlit or FastAPI + React) that:

- Accepts student responses to DASS-21.
- Returns risk category + confidence + explainability.
- Shows dashboard with historical trends.
- Implements validity checks (flags fake responses).

✓ **ML model** that:

- Achieves 70–80% accuracy (honest reporting).
- Performs ~equally across demographics (fairness tested).
- Is interpretable (feature importance visible).
- Generalizes (external validation if real data).

✓ **Written report** (10–15 pages) covering:

- Literature review + related work.
- System design & ethical considerations.
- Model development & evaluation.
- Limitations & recommendations.
- All code on GitHub.

✅ **Working demo** + presentation (5–7 min):

- Live walkthrough.
 - Explain design decisions (esp. fraud detection + ethics).
 - Discuss lessons learned.
-

Key Reminders

🎯 **Scope Carefully:** Don't try to do everything. Variant B (questionnaire + behavioral + basic fraud) is excellent for 1 semester.

🎯 **Focus on Ethical Design:** Your main differentiator vs. existing work. Emphasize throughout.

🎯 **Be Honest About Accuracy:** If model gets 70%, say 70%. Don't oversell. This builds credibility.

🎯 **Involve Your Advisor Early:** Weekly meetings week 1 onward. Get feedback on scope, ethics, progress.

🎯 **Test with Real Users (If Possible):** Even 20–30 real student responses validates your model far more than 1000 synthetic ones.

🎯 **Document Everything:** Make it easy for others to replicate or build upon your work.

Sample Outline for Your First Mentor Meeting

Come prepared with:

1. Problem statement (1 slide):

- "Rising student mental health crises + existing tools lack fraud detection + ethical design."

2. Proposed solution (1 slide):

- "ML web app combining questionnaire + behavioral signals + fraud detection."

3. Scope (Variant A/B/C) with timeline (1 slide).

4. Questions:

- "Is external validation on real data feasible within ethics/timeline?"
- "Should I prioritize breadth (more features) or depth (fewer features, well-done)?"
- "What are your thoughts on my planned tech stack?"

You've got this. Build something that matters. 🚀

Appendix: Quick Reference

Key Papers:

1. Nishimura et al. (2023) – JMIR – Multi-year ML prediction with answering time.
2. Scielosp (2024) – Systematic review – 47 ML studies; low evidence quality.
3. PMC (2023) – Online survey fraud detection – Best practices.

Questionnaire:

- DASS-21 (free, public domain, 21 items, 0–3 Likert scale).

ML Model:

- Random Forest (balanced, interpretable, no deep learning needed).

Tech Stack:

- **MVP:** Streamlit (Python only; 4 weeks).
- **Production:** FastAPI (backend) + React (frontend) + PostgreSQL (database).

Key Metrics:

- Accuracy, F1, ROC-AUC.
- Demographic Parity Gap (fairness).
- Fraud detection rate.

Main Ethical Issues:

- Misclassification, privacy, stigma, bias, scope creep, over-reliance.
-

End of Comprehensive Guide.

Questions? Feedback? This guide is yours to iterate on. Adapt as you learn more. Good luck! 🎓